"Intel Quick Sync Video" is the marketing name for a set of hardware features available inside many Intel GPUs.

Hardware Support ¶

Platform Name	Graphics	Adds support for...
Ironlake	gen5	MPEG-2, H.264 decode.
Sandy Bridge	gen6	VC-1 decode; H.264 encode.
Ivy Bridge	gen7	JPEG decode; MPEG-2 encode.
Bay Trail	gen7	-
Haswell	gen7.5	-
Broadwell	gen8	VP8 decode.
Braswell	gen8	H.265 decode; JPEG, VP8 encode.
Skylake	gen9	H.265 encode.
Apollo Lake	gen9	VP9, H.265 Main10 decode.
Kaby Lake	gen9.5	VP9 profile 2 decode; VP9, H.265 Main10 encode.
Coffee Lake	gen9.5	-
Gemini Lake	gen9.5	-
Cannon Lake	gen10	-
Ice Lake	gen11	H.265 8 bits 422/444 decode,encode; vp9 8/10 bits 444 decode,encode
Tiger Lake	gen12	av1 8,10 bits; H.265 12 bits decode;vp9 12 bits decode
Rocket Lake	gen12	-
DG1	gen12	vp8 encode removed
SG1	gen12	-
Alder Lake	gen12	-
DG2	gen12	AV1 encode.
Raptor Lake	gen12	-

(Each new platform supports a superset of the capabilities of the previous platform. you can find detailed information here)

Quality and performance benchmark:

ATS-M: here DG1: here

API Support

The hardware can be accessed through a number of different APIs:

libvpl on Linux / Windows

> This is a library from Intel which can be installed as part of Intel oneAPI Video Processing Library (OneVPL). OneVPL is the successors of MSDK and will support more encode and decode cases. spec: https://spec.oneapi.io/versions/latest/elements/oneVPL/source/index.html

VAAPI with iHD driver

> The newer VAAPI driver variant from Intel which is also used by libmfx and libvpl on Linux. There's a completely free distribution and one that includes closed-source kernels.

DXVA2 / D3D11VA

> These are standard Windows APIs, which are implemented by the Intel graphics driver to support video decode.

libmfx on Linux / Windows

> This is a library from Intel which can be installed as part of the Intel Media SDK, and supports a subset of encode and decode cases.

VAAPI with i965 driver

> This is a mostly-free (but see below) driver for the libva / VAAPI instructure. Most Linux distributions package it.

Media Foundation

> Another Windows API which supports some encode and decode cases via the Intel graphics drivers. Encode of H.264, HEVC, AAC, AC3 and MP3 supported since version 4.3.

Linux

A whole open source media stack is provided with much wider HW platforms and Linux distributions supported.
(You can also download the installation binary Media Server Studio from https://software.intel.com/en-us/intel-media-server-studio/.
However, only limited HW platforms and Linux distributions are supported by Media Server Studio.)

Intel open source media stack

Project Name	Supported Gen Graphics	Open Source Repo
oneVPL	gen12+	https://github.com/oneapi-src/oneVPL
iHD driver	gen8+	https://github.com/intel/media-driver
Libva	gen5+	https://github.com/intel/libva
MSDK	gen8 ~ gen12(Rocket Lake)	https://github.com/Intel-Media-SDK/MediaSDK
i965 driver	gen5 ~ gen9.5	https://github.com/intel/intel-vaapi-driver

Note: For MSDK and oneVPL, the supported Gen Graphics listed here is for runtime. The dispatcher for each of them can load others runtime.

i965 vs. iHD Driver vs. libvpl / libmfx

iHD Driver

- Supports VAAPI, libmfx (MSDK) and libvpl (oneVPL)
- Interoperable with OpenCL and Intel OpenCL extensions

iHD + libvpl (oneVPL) / libmfx (MSDK)

- May give better encode quality in some cases (such as look_ahead).
- Provides higher encode throughput in many cases
- Wider codec support
- Common API for applications which may also run on Windows.

i965 Driver

- Supports VAAPI
- i965 is packaged as standard in most Linux distributions.
- Runs on older and cheaper devices.
- Common API for applications which may also use AMD hardware with Mesa.
- Interoperable with standard APIs (EGL/OpenGL, OpenCL).

Windows

The MSDK runtime is part of the Intel Graphics drivers. Installation of the SDK is not required on Windows.

The SDK might still be useful for reference and documentation: https://software.intel.com/media-sdk

Running

VAAPI

See Hardware/VAAPI.

DXVA2 / D3D11VA

libvpl

The library is the successors of libmfx. It will support Intel's platform >= Alder Lake and will add new features. libvpl.so is compatible with MSDK's runtime (libmfxhw64.so) and libmfx.so also has forward compatibility with oneVPL's runtime (libmfx-gen.so).

libmfx

The library has a large number of options to set, the possible valid values of are dependent on the version and hardware. libavcodec attempts to map common options sensibly to the libmfx options, but the mapping is crude and has holes, especially around rate control.

Installing the oneVPL / Media SDK on Linux

Install oneVPL

Install dispatcher:

- Follow the guide of https://github.com/oneapi-src/oneVPL

Install runtime:

- Follow the guide of https://github.com/oneapi-src/oneVPL-intel-gpu

Install with open source MSDK stack

Install MSDK and iHD driver:

- Follow the guide of https://github.com/Intel-Media-SDK/MediaSDK to build from source code.

- Starting from Ubuntu 19.04, it is possible to install via apt (see: https://github.com/Intel-Media-SDK/MediaSDK/wiki/Intel-media-stack-on-Ubuntu):

```
sudo apt-get install libva-dev libmfx-dev
```

Build FFmpeg with oneVPL

```
./configure --enable-libvpl && make -j8 && make install
```

Build FFmpeg with MSDK

```
./configure --enable-libmfx && make -j8 && make install
```

Licence status of i965 VAAPI driver

All of the user source code is available, but it includes proprietary blobs of compiled GPU code. Since the complete human-readable source is not available, this certainly renders it GPL-incompatible and is likely to cause issues with other copyleft licences. Using the libva dynamic-loading shim mostly sidesteps this, and therefore is encouraged.

Full Examples

For detailed parameter description, please refer to the documentation. QSV Decoder, QSV Encoder

Decode-only

Check supported qsv decoder list

```
ffmpeg -decoders|grep qsv
```

H264 video decode and download as yuv420p raw file

```
ffmpeg -hwaccel qsv -c:v h264_qsv -i input.mp4 -vf hwdownload,format=nv12 -pix_f
```

H264 video decode and display with sdl

```
ffmpeg -hwaccel qsv -c:v h264_qsv -i input.mp4 -vf hwdownload,format=nv12 -pix_f
```

H264 video decode without output (this can be used as a crude benchmark of the decoder)

```
ffmpeg -hwaccel qsv -c:v h264_qsv -i input.mp4 -f null -
```

HEVC 10bit video decode and download as p010le yuv file

```
ffmpeg -hwaccel qsv -c:v hevc_qsv -load_plugin hevc_hw -i input.mp4 -vf hwdownlo
```

Encode-only

Check supported qsv encoder list

```
ffmpeg -encoders|grep qsv
```

Check private option list of h264 encoder

```
ffmpeg -h encoder=h264_qsv
```

Encode a 1080p raw yuv input as H264 with 5Mbps using VBR mode

```
ffmpeg -init_hw_device qsv=hw -filter_hw_device hw -f rawvideo -pix_fmt yuv420p
```

Encode a 1080p p010le raw yuv input as HEVC main10 profile (supported since Kaby Lake platform)

```
ffmpeg -init_hw_device qsv=hw -filter_hw_device hw -v verbose -f rawvideo -videc
-vf 'hwupload=extra_hw_frames=64,format=qsv' -c:v hevc_qsv -profile:v main10 out
```

Transcode

Software decode + h264 qsv encode with 5Mbps using CBR mode

```
ffmpeg -init_hw_device qsv=hw -filter_hw_device hw -i input.mp4 -vf hwupload=ext
```

Software decode + h264 qsv encode with CQP mode

```
ffmpeg -init_hw_device qsv=hw -filter_hw_device hw -i input.mp4 -vf hwupload=ext
```

H264 qsv decode + h264 qsv encode with 5Mbps using VBR && Look_ahead mode (look_ahead may not be supported on some platforms)

```
ffmpeg -hwaccel qsv -c:v h264_qsv -i input.mp4 -c:v h264_qsv -b:v 5M -look_ahead
```

As previous, but use ICQ mode (which is similar to crf mode of x264)

```
ffmpeg -hwaccel qsv -c:v h264_qsv -i input.mp4 -c:v h264_qsv -global_quality 25
```

As previous, but use ICQ && Look_ahead mode

```
ffmpeg -hwaccel qsv -c:v h264_qsv -i input.mp4 -c:v h264_qsv -global_quality 25
```

Hevc qsv decode + qsv scaling to 1080p@60fps + h264 qsv encode

```
ffmpeg -hwaccel qsv -c:v hevc_qsv -i input.mp4 -vf 'vpp_qsv=framerate=60,scale_
```

Hevc qsv decode + qsv scaling to 1080p@60fps and color space convert from nv12 to rgb32 + output as sdl

```
ffmpeg -hwaccel qsv -c:v hevc_qsv -i input.mp4 -vf 'vpp_qsv=framerate=60,scale_
```

1:N transcoding: (MFE will be enabled by default if MSDK can support it)

```
ffmpeg -hwaccel qsv -c:v h264_qsv -i input.mp4 \
     -filter_complex "split=2[s1][s2]; [s1]scale_qsv=1280:720[o1];[s2]scale_
     -map [o1] -c:v h264_qsv -b:v 3200k 3200a.mp4 \
     -map [o2] -c:v h264_qsv -b:v 1750k 1750a.264
```

M:N transcoding

```
ffmpeg -hwaccel qsv -c:v h264_qsv -i input1.mp4 -hwaccel qsv -c:v h264_qsv -i
  -filter_complex '[0:v]split=2[out1][out2],[1:v]split=2[out3][out4]' \
  -map '[out1]' -c:v h264_qsv output1.mp4 -map '[out2]' -c:v h264_qsv output2
  -map '[out3]' -c:v h264_qsv output3.mp4 -map '[out4]' -c:v h264_qsv output4
```

`-qsv_device` is an qsv customized option can be used to specify a hardware device and avoid the default device initialization failure when multiple devices usable (eg: an Intel integrated GPU and an AMD/Nvidia discrete graphics card). One example on Linux (more details please see https://trac.ffmpeg.org/ticket/7649)

```
ffmpeg -hwaccel qsv -qsv_device /dev/dri/renderD128 -c:v h264_qsv -i input.mp4 -
```

Hybrid transcode

It is also possible to use "vaapi decode + vaapi scaling + qsv encode" (available on Linux platform)

```
ffmpeg -hwaccel vaapi -hwaccel_output_format vaapi -i input.mp4 -vf 'scale_vaapi
```

Or use "dxva decode + qsv scaling + qsv encode" (available on Windows)

```
ffmpeg -hwaccel dxva2 -hwaccel_output_format dxva2_vld -i input.mp4 -vf 'hwmap=d
```

Last modified on Sep 29, 2024, 7:03:44 AM